
# Universidad Nacional Autónoma de México

Computer Engineering

Databases

# Restaurant Management Database System

## Final Project

**Capitos Dream Team**

**Students:**

- Alvarez Salgado Eduardo Antonio
- Lara Hernández Emmanuel
- Merida Serralde Francisco Jared
- Ponce de León Reyes Bruno
- Zepeda Aparicio Diego Arturo

Group: 1
Semester: 2026-II

Mexico City, Mexico. May 2026

# Contents

# 1. Introduction

## 1.1 Problem Statement

A local restaurant requires a digital transformation of its daily operations through the development of a comprehensive, multi-module IT system. The core of this transformation relies on a robust database capable of managing complex, interconnected operational data. The system must maintain detailed employee records, including personal information, contact details, specific roles, and information regarding their dependents. Furthermore, the database must accurately track the restaurant's inventory of food and beverages, categorizing each item and monitoring its recipe, price, and current availability.

At the transactional level, the system needs to record detailed order information, including generated folios, precise timestamps, assigned staff, and itemized billing details. Because customers may request formal invoices, the database must also securely store client billing information. Finally, the restaurant operates a secondary database that requires daily updates via text files, creating an additional need for a reliable data ingestion and error-handling process.

## 1.2 Objectives

- The student will analyze a series of requirements and propose a solution that addresses them, applying the concepts learned in the course.
- Design and implement a robust relational database schema for a Restaurant Management System.
- Develop database-level programming components, including stored procedures, triggers, and views using PostgreSQL, to automate business logic and maintain data integrity.

## 1.3 Proposed Solution

To address the restaurant's operational needs, we propose the design and implementation of a relational database using PostgreSQL (PgSQL), ensuring compliance with all structural design principles and data management best practices. The solution will leverage database-level programming to automate critical business rules and maintain data integrity. Specifically, we will implement database triggers to automatically validate product availability and dynamically calculate order totals in real time as items are added. To facilitate business intelligence and reporting, we will create custom SQL views that can instantly identify the best-selling dish and automatically

generate an invoice-style summary for any given order.
Additionally, the system will use stored procedures and functions to track employee performance, such as verifying a server's daily order count and revenue generated, tracking out-of-stock inventory, generating invoice data, and calculating total sales figures within specific date ranges. Database performance will be optimized through strategic indexing based on anticipated query loads. Finally, to meet the secondary database migration requirement, we will develop a structured data-loading script that coordinates the insertion of information from text files into the secondary database, with robust flow validation to handle potential insertion errors.

# 2. Work Plan

## 2.1 Activity Plan

| Stage | Activity | Responsible Member(s) |
|---|---|---|
| 1 | **Requirement analysis**: Reviewing the project specifications to identify entities, attributes, business rules, and system constraints required by the restaurant. | Team |
| 2 | **Conceptual database design:** Creating the Entity-Relationship model to visually represent the database structure, including relationships between employees, orders, products, and customers. | Alvarez Salgado Eduardo Antonio / Zepeda Aparicio Diego Arturo |
| 3 | **Relational model design:** Translating the conceptual model into a relational schema by defining tables, primary keys, foreign keys, and data normalization rules. | Alvarez Salgado Eduardo Antonio / Merida Serralde Francisco Jared |
| 4 | **DDL script creation:** Writing the Data Definition Language (DDL) code in PostgreSQL to physically create the tables, sequences, domains, and constraints within the database. | Ponce de León Reyes Bruno |

| | | |
|---|---|---|
| 5 | **Initial data loading script:** Developing the Data Manipulation Language (DML) script to insert the initial sample records and populate the database for testing. | Ponce de León Reyes Bruno / Lara Hernández Emmanuel |
| 6 | **Database-level programming:** Implementing triggers, stored procedures, functions, and views to automate business logic, update order totals, and ensure data integrity. | Merida Serralde Francisco Jared / Ponce de León Reyes Bruno / Alvarez Salgado Eduardo Antonio |
| 7 | **Presentation layer implementation:** Establishing the connection method to the database and setting up the mechanism (such as a command-line interface or application) for user interaction. | Lara Hernández Emmanuel / Zepeda Aparicio Diego Arturo |
| 8 | **Documentation and final review:** Compiling the formal report and performing a final verification of the database functionality against the initial problem statement. | Team |

## 2.2 Team Work Distribution

- **Alvarez Salgado Eduardo Antonio** — Database modeler, designer and developer.
- **Lara Hernández Emmanuel** — Data loading specialist and presentation layer developer.
- **Merida Serralde Francisco Jared** — Database developer and relational model designer.
- **Ponce de León Reyes Bruno** — DDL/DML script creator and database-level programmer.
- **Zepeda Aparicio Diego Arturo** — Database conceptual modeler and presentation layer developer.

# 3. Design

## 3.1 Conceptual Model

Once the requirements for creating the database were analyzed, the conceptual design phase began. The importance of this phase lies in the fact that it makes it possible to understand and organize the information involved in the problem, allowing the project to follow a clear structure and facilitating the development of the following phases.

For this purpose, an Extended Entity-Relationship Model (EER) was created using Chen notation, with the support of the DIA modeling software. Based on the analysis of the problem statement, the following main entities were identified:

- Empleado
- Dependiente_Empleado
- Cocinero
- Mesero
- Administrativo
- Orden
- Producto
- Categoría
- Cliente
- Factura

EMPLEADO is one of the central elements of the design, since it stores the general information of the restaurant workers. From this entity, a total and non-exclusive specialization, also known as overlapping specialization, was defined toward the subtypes COCINERO, MESERO, and ADMINISTRATIVO. It is non-exclusive because the problem statement indicates that an employee may have more than one position, meaning that the same worker could be, for example, a cook and a waiter at the same time. This decision avoids repeating the general data of employees and allows the model to store only the specific attributes of each position: specialty for cooks, schedule for waiters, and role for administrative employees.

The entity Dependiente_Empleado is directly related to Empleado through the relationship tiene. It was modeled as a weak entity by identification, since dependents are registered according to the employee with whom they are associated.

The entity Orden represents the orders placed in the restaurant. It stores the order folio, the date and time when the order was created, and the total amount to be paid. The latter was represented as a derived attribute, since it is obtained from the sum of the amounts of the products included in the order. This entity is related to MESERO so that the identifier of the waiter who registered the order is stored.

On the other hand, the relationship between Orden and Producto was represented through the relationship contiene, with M:M cardinality, which specifies the quantity and the total price per product.

The entity PRODUCTO stores the information of the dishes and beverages offered by the restaurant. Although there are two types of products, they were modeled within a single entity and distinguished through the attribute tipoProducto.

This is because both types share the same common attributes and have the same relationships with other entities in the database. Although it would have been possible to represent dishes and beverages as subtypes through a generalization, for the purposes of this exercise, where no specific attributes are stored for each one, it was decided to keep them as a single entity. In this way, the model remains simpler and avoids creating subtypes without their own information.

The entity CATEGORÍA was kept as an independent entity and is directly related to PRODUCTO, allowing products to be classified according to their corresponding category.

Finally, the entities CLIENTE and FACTURA were included. The entity CLIENTE is not directly related to ORDEN because not every order requires customer information to be stored. Instead, customer data are only recorded when an invoice is requested.

For this reason, the entity FACTURA was created as an intermediate element between CLIENTE and ORDEN. The relationship solicita indicates that a customer may request zero or many invoices, while each invoice belongs to only one customer. Likewise, the relationship genera indicates that an order may generate zero or one invoice, while each invoice must correspond to exactly one order. This represents the optional nature of invoicing and prevents the system from storing customer data when no invoice is requested.

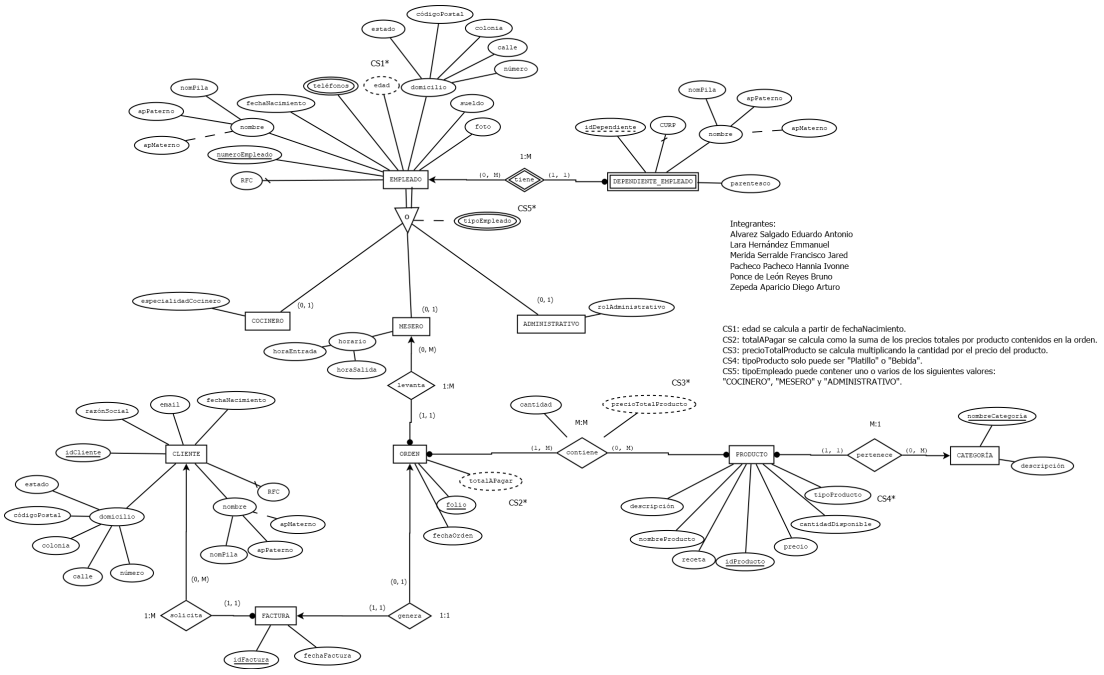



Figure 1: Conceptual Entity-Relationship Model

## 3.2 Relational Model

Once the conceptual design phase was completed, the logical design phase was carried out. In this phase, the Extended Entity-Relationship Model was transformed into a relational model using crow's foot notation. The importance of this phase lies in the fact that it makes it possible to convert the previously identified entities, attributes, and relationships into a structure based on tables, primary keys, foreign keys, constraints, and cardinalities, bringing the design closer to a future implementation in a database management system.

The main objective of the relational model was to preserve the logic defined in the conceptual design while adapting it to a structure that could later be implemented. To achieve this, the corresponding transformation rules were applied, taking into account the type of entity, the cardinality of each relationship, and the nature of the attributes.

First, a table was created for each main entity identified in the conceptual model. Composite attributes, such as name and address, were decomposed into individual attributes, allowing the information to be stored more precisely.

Multivalued attributes were also transformed into independent tables. This was the case for phone numbers, which belong to EMPLEADO, since a single employee may have more than one phone number. Therefore, the table TELEFONO_EMPLEADO was created and related to EMPLEADO through a foreign key.

Finally, for the many-to-many relationship between ORDEN and PRODUCTO, the intermediate table DETALLE_ORDEN was created. This table makes it possible to represent the products included in each order and to store the attributes that belong to the relationship itself, such as the quantity and the total price per product.

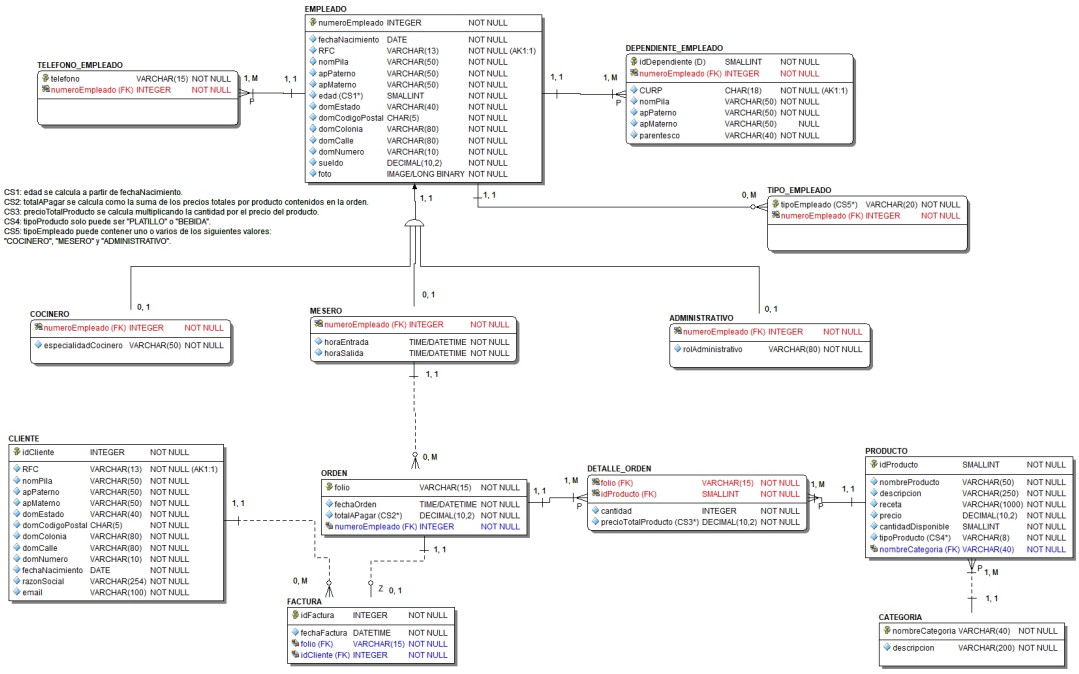



Figure 2: Conceptual Entity-Relationship Model

# 4. Implementation

This section describes the implementation process used to create a database that satisfies the problem requirements. The implementation includes the DDL script, initial data loading, and database-level programming in PostgreSQL.

## 4.1 DDL Code

DDL (Data Definition Language) statements were used to define the database structure, including tables, constraints, primary keys, foreign keys, sequences, and views. These statements are included in the script named:

**DDL Script**

`BD_P1_DDL.sql`

The following section explains the code:

```
--Codigo DDL para generacion de tablas en la BD
--Creando tabla Categoria
CREATE TABLE CATEGORIA (
    nombreCategoria varchar(40) not null,
    descripcion varchar(200) not null,
    CONSTRAINT categoria_pk PRIMARY KEY(nombreCategoria)
);
```

The above code creates the CATEGORIA table, where "nombreCategoria" is defined as the primary key.

```
--- PRODUCTO

--Creando una secuencia para los productos
CREATE SEQUENCE seq_producto_id
START WITH 1
INCREMENT BY 1;

--Creando tabla Producto
CREATE TABLE PRODUCTO (
    idProducto smallint DEFAULT nextval('seq_producto_id'),
    nombreProducto varchar(50) not null,
    descripcion varchar(250) not null,
    receta varchar(1000) not null,
    precio decimal(10,2) not null,
    cantidadDisponible smallint not null,
    tipoProducto varchar(8) not null,
    nombreCategoria varchar(40) not null,
    CONSTRAINT producto_pk PRIMARY KEY(idProducto),
    CONSTRAINT producto_nombreCategoria_fk FOREIGN KEY(
       nombreCategoria)
        REFERENCES CATEGORIA(nombreCategoria),
    CONSTRAINT producto_cantidadDisponible_chk CHECK(
       cantidadDisponible >= 0),
    CONSTRAINT producto_tipoProducto_chk CHECK(tipoProducto
       in ('PLATILLO','BEBIDA'))
);
```

The above code creates a sequence that is used to automatically generate the ID for the PRODUCTO table. The attribute "idProducto" is defined as the primary key; additionally, the foreign keys from related tables are defined. Check constraints are

applied to verify that the quantity is greater than or equal to zero, and that product types are restricted to either "platillo" or "bebida".

```
--- CLIENTE

--Creando una secuencia para los clientes
CREATE SEQUENCE seq_cliente_id
START WITH 1
INCREMENT BY 1;

-- Creando tabla cliente
CREATE TABLE CLIENTE (
    idCliente integer DEFAULT nextval('seq_cliente_id'),
    rfc varchar(13) not null,
    nomPila varchar(50) not null,
    apPaterno varchar(50) not null,
    apMaterno varchar(50) not null,
    domEstado varchar(40) not null,
    domCodigoPostal char(5) not null,
    domColonia varchar(80) not null,
    domCalle varchar(80) not null,
    domNumero varchar(10) not null,
    fechaNacimiento date not null,
    razonSocial varchar(254) not null,
    email varchar(100) not null,
    CONSTRAINT cliente_pk PRIMARY KEY (idCliente),
    CONSTRAINT cliente_rfc_uk UNIQUE(rfc)
);
```

The above code creates a sequence that is used to automatically generate the ID for the CLIENTE table. The attribute "idCliente" is defined as the primary key. A unique constraint is applied to the attribute "rfc" to ensure that it does not repeat.

```
--- EMPLEADO

--Creando una secuencia para los empleados
CREATE SEQUENCE seq_empleado_num
START WITH 1
INCREMENT BY 1;
```

```
--Creando tabla empleado
CREATE TABLE EMPLEADO (
    numeroEmpleado integer DEFAULT nextval('seq_empleado_num'
       ),
    fechaNacimiento date not null,
    rfc varchar(13) not null,
    nomPila varchar(50) not null,
    apPaterno varchar(50) not null,
    apMaterno varchar(50) not null,
    -- Se calculara la edad con el uso de una Vista
    domEstado varchar(40) not null,
    domCodigoPostal char(5) not null,
    domColonia varchar(80) not null,
    domCalle varchar(80) not null,
    domNumero varchar(10) not null,
    sueldo decimal(10,2) not null,
    foto bytea not null,
    CONSTRAINT empleado_pk PRIMARY KEY(numeroEmpleado),
    CONSTRAINT empleado_rfc_uk UNIQUE(rfc)
);
```

The above code creates a sequence that is used to automatically generate the ID for the EMPLEADO table. The attribute "numeroEmpleado" is defined as the primary key. A unique constraint is applied to the attribute "rfc" to ensure that it does not repeat.

```
-- Creacion de la vista v_empleado que incluye la edad
   calculada automaticamente
CREATE VIEW v_empleado AS
SELECT
    numeroEmpleado,
    fechaNacimiento,
    rfc,
    nomPila,
    apPaterno,
    apMaterno,
    cast(extract(year from age(fechaNacimiento)) as smallint)
        as edad,
    domEstado,
    domCodigoPostal,
    domColonia,
```

```
    domCalle,
    domNumero,
    sueldo,
    foto
FROM EMPLEADO;
```

The above code creates a view that is used to display all the attributes from the EMPLEADO table and automatically calculate the employee's age.

```
--Creando tabla Telefono_Empleado
CREATE TABLE TELEFONO_EMPLEADO (
    telefono varchar(15) not null,
    numeroEmpleado integer not null,
    CONSTRAINT telefono_empleado_pk PRIMARY KEY(telefono,
       numeroEmpleado),
    --Se utilizo on delete cascade por si se elimina un
       registro de empleado,tambien se eliminen sus telefonos
        asocaidos
    CONSTRAINT telefono_empleado_numeroEmpleado_fk FOREIGN
       KEY(numeroEmpleado)
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
);
```

The above code creates the TELEFONO_EMPLEADO table. The attributes "telefono" and "numeroEmpleado" are defined as the primary keys. This table represents the multivalued attribute "telefono" of an employee.

```
--Creando tabla dependiente_empleado
CREATE TABLE DEPENDIENTE_EMPLEADO (
    idDependiente smallint not null,
    numeroEmpleado integer not null,
    curp char(18) not null,
    nomPila varchar(50) not null,
    apPaterno varchar(50) not null,
    apMaterno varchar(50) null,
    parentesco varchar(40) not null,
    CONSTRAINT dependiente_empleado_pk PRIMARY KEY(
       idDependiente, numeroEmpleado),
    CONSTRAINT dependiente_empleado_numeroEmpleado_fk FOREIGN
        KEY(numeroEmpleado)
```

```
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
            ,
    CONSTRAINT dependiente_empleado_curp_uk UNIQUE(curp)
);
```

The above code creates the DEPENDIENTE_EMPLEADO table. The attributes "idDependiente" and "numeroEmpleado" are defined as the primary keys because this table is a weak entity; additionally, the primary key from the CLIENTE table is defined as a foreign key. A unique constraint is applied to the attribute "curp".

```
-- Creando tabla tipo_empleado
CREATE TABLE TIPO_EMPLEADO (
    tipoEmpleado varchar(20) not null,
    numeroEmpleado integer not null,
    CONSTRAINT tipo_empleado_pk PRIMARY KEY(tipoEmpleado,
       numeroEmpleado),
    CONSTRAINT tipo_empleado_numeroEmpleado_fk FOREIGN KEY(
       numeroEmpleado)
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
            ,
    CONSTRAINT tipo_empleado_tipoEmpleado_chk CHECK(
       tipoEmpleado in ('COCINERO','MESERO','ADMINISTRATIVO')
       )
);
```

The above code creates the TIPO_EMPLEADO table. The attributes "tipoEmpleado" and "numeroEmpleado" are defined as the primary keys because this table is a weak entity; additionally, the primary key from the CLIENTE table is defined as a foreign key. A check constraint is applied to ensure that the employee type is either "COCINERO", "MESERO" or "ADMINISTRATIVO".

```
--Creando tabla de cocinero
CREATE TABLE COCINERO (
    numeroEmpleado integer not null,
    especialidadCocinero varchar(50) not null,
    CONSTRAINT cocinero_pk PRIMARY KEY(numeroEmpleado),
    CONSTRAINT cocinero_numeroEmpleado_fk FOREIGN KEY(
       numeroEmpleado)
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
);
```

The above code creates the COCINERO table. This table is a subtype of the EMPLEADO table; the attribute "numeroEmpleado" is defined as a foreign key and as the primary key because the COCINERO table is a weak entity.

```
--Creando tabla de mesero
CREATE TABLE MESERO (
    numeroEmpleado integer not null,
    horaEntrada time not null,
    horaSalida time not null,
    CONSTRAINT mesero_pk PRIMARY KEY(numeroEmpleado),
    CONSTRAINT mesero_numeroEmpleado_fk FOREIGN KEY(
       numeroEmpleado)
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
);
```

The above code creates the MESERO table. This table is a subtype of the EMPLEADO table; the attribute "numeroEmpleado" is defined as a foreign key and as the primary key because the MESERO table is a weak entity.

```
--Creando tabla de administrativo
CREATE TABLE ADMINISTRATIVO (
    numeroEmpleado integer not null,
    rolAdministrativo varchar(80) not null,
    CONSTRAINT administrativo_pk PRIMARY KEY(numeroEmpleado),
    CONSTRAINT administrativo_numeroEmpleado_fk FOREIGN KEY(
       numeroEmpleado)
        REFERENCES EMPLEADO(numeroEmpleado) ON DELETE CASCADE
);
```

The above code creates the ADMINISTRATIVO table. This table is a subtype of the EMPLEADO table; the attribute "numeroEmpleado" is defined as a foreign key and as the primary key because the ADMINISTRATIVO table is a weak entity.

```
--- ORDEN

--Creando una secuencia para las ordenes
CREATE SEQUENCE seq_orden_folio
START WITH 1
INCREMENT BY 1;
```

```
--Creando tabla de orden
CREATE TABLE ORDEN (
    folio varchar(15) DEFAULT ('ORD-' || to_char(nextval('
       seq_orden_folio'),'FM0000')),
    fechaOrden timestamp not null,
    totalAPagar decimal(10,2) DEFAULT 0 not null, --Se hace
       uso de trigger para calcular el pago total
    numeroEmpleado integer not null,
    CONSTRAINT orden_pk PRIMARY KEY(folio),
    CONSTRAINT orden_numeroEmpleado_fk FOREIGN KEY(
       numeroEmpleado)
        REFERENCES MESERO(numeroEmpleado)
);
```

The above code creates a sequence that is used to automatically generate the ID for the ORDEN table. The attribute "folio" is defined as the primary key and follows the specified format 'ORD-0000'. In addition, foreign keys referencing related tables are defined. The value of the attribute "totalAPagar" is automatically calculated by a trigger when a record is inserted into the table.

```
--- FACTURA

--Creando una secuencia para las facturas
CREATE SEQUENCE seq_factura_id
START WITH 1
INCREMENT BY 1;

--Creando tabla de factura
CREATE TABLE FACTURA (
    idFactura integer DEFAULT nextval('seq_factura_id'),
    fechaFactura timestamp not null,
    folio varchar(15) not null,
    idCliente integer not null,
    CONSTRAINT factura_pk PRIMARY KEY(idFactura),
    CONSTRAINT factura_folio_fk FOREIGN KEY(folio)
        REFERENCES ORDEN(folio),
    CONSTRAINT factura_idCliente_fk FOREIGN KEY (idCliente)
        REFERENCES CLIENTE(idCliente)
);
```

The above code creates a sequence that is used to automatically generate the ID

for the FACTURA table. The attribute "idFactura" is defined as the primary key; additionally, the primary key attributes of the CLIENTE and ORDEN tables are referenced as foreign keys.

```
--Creando tabla de detalle_orden
CREATE TABLE DETALLE_ORDEN (
    folio varchar(15) not null,
    idProducto smallint not null,
    cantidad integer not null,
    precioTotalProducto decimal(10,2) DEFAULT 0 not null, --
       Se hace uso de un trigger para calcular el total
    CONSTRAINT detalle_orden_pk PRIMARY KEY(folio, idProducto
       ),
    CONSTRAINT detalle_orden_folio FOREIGN KEY(folio)
        REFERENCES ORDEN(folio) ON DELETE CASCADE,
    CONSTRAINT detalle_orden_idProducto_fk FOREIGN KEY (
       idProducto)
        REFERENCES PRODUCTO(idProducto)
);
```

The above code creates the DETALLE_ORDEN table. The attributes "folio" and "idProducto" are defined as the primary keys. In addition, foreign keys referencing related tables are defined. The value of the attribute "precioTotalProducto" is automatically calculated by a trigger when a record is inserted or updated.

## 4.2 DML Code: Initial Data Loading

This section describes the script used to insert the initial data into the database:

**DML Script**

BD_P1_DML.sql

```
--Codigo DML para la carga de registros iniciales

--- CATEGORIA
insert into CATEGORIA(nombreCategoria,descripcion)
    values ('ENTRADA','Platillo para empezar'),
           ('PLATO_FUERTE','Platillo principal'),
           ('POSTRE','Platillo para el final'),
```

```
            ('BEBIDA_FRIA','Bebidas frescas'),
            ('BEBIDA_CALIENTE','Bebidas preparadas (cafes,
               chocolates,etc.)');

--- PRODUCTO
insert into PRODUCTO(nombreProducto, descripcion, receta,
   precio, cantidadDisponible, tipoProducto, nombreCategoria)
    values ('Bruschetta Clasica', 'Pan tostado con tomate
       fresco, ajo, albahaca y aceite de oliva', 'Pan
       ciabatta, tomates, ajo, albahaca, aceite de oliva',
       95.00, 12, 'PLATILLO', 'ENTRADA'),
           ('Carpaccio de Res', 'Laminas de filete de res con
               alcaparras, parmesano y arugula', 'Filete de
              res, alcaparras, queso parmesano, aceite',
              165.00, 10, 'PLATILLO', 'ENTRADA'),
           ('Lasana Bolonesa', 'Capas de pasta con carne,
              salsa bechamel y queso gratinado', 'Laminas de
              pasta, carne molida, jitomate, bechamel,
              mozzarella', 195.00, 7, 'PLATILLO', '
              PLATO_FUERTE'),
           ('Spaghetti alla Carbonara', 'Pasta tradicional
              con huevo, queso y pimienta', 'Spaghetti, huevo
              , queso pecorino romano, guanciale, pimienta',
              170.00, 8, 'PLATILLO', 'PLATO_FUERTE'),
           ('Pizza Margherita', 'Salsa de tomate, mozzarella
              fresca y albahaca', 'Masa de pizza, salsa de
              tomate San Marzano, mozzarella, albahaca',
              155.00, 9, 'PLATILLO', 'PLATO_FUERTE'),
           ('Tiramisu', 'Postre en capas de soletas, cafe y
              crema Mascarpone', 'Soletas, cafe, queso
              mascarpone, huevo, cacao', 90.00, 6, 'PLATILLO'
              , 'POSTRE'),
           ('Panna Cotta de Frutos Rojos', 'Postre frio con
              dulce de fresas y frambuesas', 'Crema para
              batir, azucar, grenetina, coulis de frutos
              rojos', 75.00, 11, 'PLATILLO', 'POSTRE'),
           ('Capuccino', 'Cafe espresso con leche y canela',
              'Espresso, leche entera, canela', 55.00, 16, '
              BEBIDA', 'BEBIDA_CALIENTE');

--- CLIENTE
```

```
insert into CLIENTE(rfc, nomPila, apPaterno, apMaterno,
   domEstado, domCodigoPostal, domColonia, domCalle,
   domNumero, fechaNacimiento, razonSocial, email)
    values ('SAPA900115A10', 'Alejandro', 'Sanchez', '
       Palacios', 'CDMX', '03100', 'Del Valle', 'Av. Coyoacan
       ', '1024', '1990-01-15', 'Alejandro Sanchez Palacios',
        'alejandro.san@mail.com'),
           ('GOMM851122B20', 'Monica', 'Gomez', 'Mendoza', '
              EdoMex', '53100', 'Satelite', 'Circuitos
              Actores', '45', '1985-11-22', 'Gomez y
              Asociados S.A.', 'monica.gomez@mail.com'),
           ('LORE930504C30', 'Eduardo', 'Lopez', 'Ramirez', '
              CDMX', '06700', 'Roma Norte', 'Alvaro Obregon',
               '88', '1993-05-04', 'Eduardo Lopez Ramirez', '
              eduardo.lop@mail.com'),
           ('FERF970812D40', 'Fernanda', 'Fernandez', 'Flores
              ', 'Jalisco', '44100', 'Americana', 'Av.
              Libertad', '310', '1997-08-12', 'Fernanda
              Fernandez Flores', 'fer.flores@mail.com'),
           ('BIN051020TL4', 'Bienes', 'Raices', '
              Inmobiliarios', 'CDMX', '06600', 'Juarez', '
              Paseo de la Reforma', '222', '2005-10-20', '
              Bienes Inmobiliarios del Norte S.A. de C.V.', '
              contacto@binorte.com');

--- EMPLEADO
insert into EMPLEADO(fechaNacimiento, rfc, nomPila, apPaterno
   , apMaterno, domEstado, domCodigoPostal, domColonia,
   domCalle, domNumero, sueldo, foto)
    values ('1992-03-10', 'HELR920310AA1', 'Roberto', '
       Hernandez', 'Luna', 'CDMX', '03100', 'Del Valle', '
       Amores', '412', 16000.00, '\x00'),
           ('1995-07-22', 'GARM950722BB2', 'Maria', 'Garcia',
               'Moreno', 'CDMX', '06700', 'Roma Norte', '
              Orizaba', '93', 11000.00, '\x00'),
           ('1988-11-05', 'PEAL881105CC3', 'Luis', 'Perez', '
              Alba', 'EdoMex', '53100', 'Satelite', 'Circuito
               Juristas', '14', 18500.00, '\x00'),
           ('1994-05-14', 'MORE940514DD4', 'Elena', 'Morales'
              , 'Rios', 'CDMX', '04000', 'Coyoacan', '
              Malintzin', '205', 11000.00, '\x00'),
```

```
            ('1991-01-29', 'SARA910129EE5', 'Arturo', 'Sanchez
               ', 'Rojas', 'CDMX', '01000', 'San Angel', '
               Juarez', '19', 14000.00, '\x00'),
            ('1996-08-18', 'CAST960818FF6', 'Tatiana', 'Castro
               ', 'Solis', 'CDMX', '07000', 'Lindavista', '
               Peten', '855', 12500.00, '\x00'),
            ('1993-12-01', 'NAVJ931201GG7', 'Juan', 'Navarro',
                'Vargas', 'EdoMex', '54000', 'Tlalnepantla', '
               Toltecas', '33', 11500.00, '\x00'),
            ('1995-02-15', 'LOMF950215KY2', 'Fernando', 'Lopez
               ', 'Martinez', 'CDMX', '09000', 'Iztapalapa', '
               Ermita', '1105', 13500.00, '\x00');

--- TELEFONO_EMPLEADO
insert into TELEFONO_EMPLEADO(telefono,numeroEmpleado)
    values ('5512345678', 1),
           ('5523456789', 2),
           ('5534567890', 3),
           ('5545678901', 4),
           ('5556789012', 5),
           ('5567890123', 6),
           ('5578901234', 7),
           ('5511223344', 8);

--- DEPENDIENTE_EMPLEADO
insert into DEPENDIENTE_EMPLEADO(idDependiente,numeroEmpleado
   ,curp,nomPila,apPaterno,apMaterno,parentesco)
    values (1, 1, 'HERR150420HDFLNR05', 'Raul', 'Hernandez',
       'Gomez', 'HERMANO'),
           (1, 3, 'PERA120805MDFLNS01', 'Alicia', 'Perez', '
               Soto', 'HIJA'),
           (1, 5, 'SANF181123HDFRRN09', 'Francisco', 'Sanchez
               ', 'Luna', 'HIJO'),
           (1, 2, 'GARM401201MDFRRN02', 'Maria', 'Moreno', '
               Cruz', 'MADRE'),
           (1, 6, 'CASS210707HDFLSS03', 'Santiago', 'Castro',
                'Diaz', 'HIJO'),
           (1, 4, 'MORL200228MDFRRS06', 'Lucia', 'Morales', '
               Vega', 'HIJA'),
           (1, 8, 'LOPD190902HDFRNT08', 'Diego', 'Lopez', '
               Ruiz', 'HIJO'),
```

```
            (2, 8, 'LOPM220614MDFRNT01', 'Mia', 'Lopez', 'Ruiz
               ', 'HIJA');

--- TIPO_EMPLEADO
insert into TIPO_EMPLEADO(tipoEmpleado,numeroEmpleado)
    values ('COCINERO', 1),
           ('MESERO', 2),
           ('ADMINISTRATIVO', 3),
           ('MESERO', 4),
           ('COCINERO', 5),
           ('COCINERO', 6),
           ('MESERO', 7),
           ('COCINERO', 8),
           ('MESERO', 8);

--- COCINERO
insert into COCINERO(numeroEmpleado,especialidadCocinero)
    values (1, 'Cortes de carne'),
           (5, 'Salsas y guarniciones'),
           (6, 'Bolleria'),
           (8, 'Entradas y Platos fuertes');

--- MESERO
insert into MESERO(numeroEmpleado,horaEntrada,horaSalida)
    values (2, '07:00:00', '15:00:00'),
           (4, '07:00:00', '15:00:00'),
           (7, '15:00:00', '23:00:00'),
           (8, '15:00:00', '23:00:00');

--- ADMINISTRATIVO
insert into ADMINISTRATIVO(numeroEmpleado,rolAdministrativo)
    values (3, 'Gerente_Compras');

--- ORDEN
insert into ORDEN(fechaOrden,numeroEmpleado)
    values ('2026-05-01 11:20:11', 2),
           ('2026-05-15 19:30:09', 4),
           ('2026-05-28 13:05:50', 7),
           ('2026-05-30 21:45:03', 2),
           ('2026-05-31 23:59:01', 8);
```

```
--- FACTURA
insert into FACTURA(fechaFactura,folio,idCliente)
    values ('2026-05-01 12:10:00', 'ORD-0001', 1),
           ('2026-05-15 20:45:00', 'ORD-0002', 2),
           ('2026-05-28 14:20:00', 'ORD-0003', 3),
           ('2026-05-30 22:30:00', 'ORD-0004', 4),
           ('2026-06-01 00:15:00', 'ORD-0005', 5);

--- DETALLE_ORDEN
insert into DETALLE_ORDEN(folio,idProducto,cantidad)
    values ('ORD-0001', 1, 2), -- 2 Bruschettas
           ('ORD-0001', 3, 1), -- 1 Lasana Bolonesa
           ('ORD-0001', 8, 2), -- 2 Capuccinos
           ('ORD-0002', 2, 1), -- 1 Carpaccio de Res
           ('ORD-0002', 4, 2), -- 2 Spaghetti Carbonara
           ('ORD-0003', 5, 1), -- 1 Pizza Margherita
           ('ORD-0003', 6, 2), -- 2 Tiramisus
           ('ORD-0004', 3, 1), -- 1 Lasana Bolonesa
           ('ORD-0004', 8, 1), -- 1 Capuccino
           ('ORD-0005', 4, 1), -- 1 Spaghetti Carbonara
           ('ORD-0005', 7, 2); -- 2 Panna Cottas
```

The above code represents all the INSERT statements used to load the initial data into the database. These statements first specify the attributes to be modified, avoiding the need to provide values for attributes that are generated automatically.

## 4.3 Database-Level Programming

This section describes the stored procedures, triggers, functions, and views implemented to satisfy the project requirements.

### 4.3.1 Triggers

Triggers are used to validate product availability and update order totals automatically when products are added to an order.

**detalle_orden and actualizar_total trigger**

```
CREATE OR REPLACE FUNCTION fun_detalle_orden()
RETURNS TRIGGER
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    v_precio NUMERIC(10,2);
    v_cantidad smallint;
    p_nombre varchar(50);
BEGIN

    SELECT precio, cantidaddisponible
    INTO v_precio, v_cantidad
    FROM producto
    WHERE idproducto = NEW.idproducto;

    IF (v_cantidad - NEW.cantidad) < 0 THEN
        SELECT p.nombreproducto INTO p_nombre FROM producto p
            JOIN detalle_orden d ON p.idproducto = d.
           idproducto WHERE p.idproducto = NEW.idproducto;
        RAISE EXCEPTION'El producto % no esta disponible o la
            cantidad deseada supera la cantidad disponible',
           p_nombre;
    END IF;

    NEW.preciototalproducto := NEW.cantidad * v_precio;
    UPDATE producto SET cantidaddisponible = (v_cantidad -
       NEW.cantidad) WHERE idproducto = NEW.idproducto;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_detalle_orden_bi
BEFORE INSERT ON detalle_orden
FOR EACH ROW
EXECUTE FUNCTION fun_detalle_orden();

CREATE OR REPLACE FUNCTION fun_actualizar_total_orden()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN

    UPDATE orden
    SET totalapagar = (
```

```
        SELECT COALESCE(SUM(preciototalproducto),0)
        FROM detalle_orden
        WHERE folio = NEW.folio
    )
    WHERE folio = NEW.folio;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_actualizar_total
AFTER INSERT
ON detalle_orden
FOR EACH ROW
EXECUTE FUNCTION fun_actualizar_total_orden();
```

The above trigger is used so that every time a product is added to an order, the product and sales totals are updated, and the system verifies that the product is still in stock.

It works by separating the product update from the sales update. In the first trigger, three variables are defined: the first two are for the product price and quantity, and the third is for the product name in case of an error. The quantity is used to determine whether the product is still available; if there are no more products, an error is sent; otherwise, the first total in the "producto" table is updated.

Finally, for the second trigger, an update is performed on the "orden" table, where the total amount due is updated using a subquery.

### 4.3.2 Stored Procedures

Stored procedures are used to execute database operations that require validation and procedural logic.

**ordenes procedure**

```
CREATE OR REPLACE PROCEDURE ordenes(num_cl int4)
AS
$$
DECLARE
```

```
    nombre varchar(50);
    ap_pat varchar(50);
    ap_mat varchar(50);
    total_ordenes int;
    total_pago numeric(10,2);

BEGIN

    SELECT nompila, appaterno, apmaterno
    INTO nombre, ap_pat, ap_mat
    FROM empleado
    WHERE numeroempleado = num_cl;

    IF EXISTS(SELECT * FROM empleado e JOIN mesero m ON e.
       numeroempleado = m.numeroempleado WHERE e.
       numeroempleado = num_cl)
      THEN
        SELECT COUNT(*)
        INTO total_ordenes
        FROM orden
        WHERE numeroempleado = num_cl AND fechaorden::date =
           CURRENT_DATE;

        SELECT SUM(totalapagar)
        INTO total_pago
        FROM orden
        WHERE numeroempleado = num_cl AND fechaorden::date =
           CURRENT_DATE;

        RAISE NOTICE 'El mesero % % % ha hecho % orden(es)
           hoy, y su total es: $%', nombre, ap_pat, ap_mat,
           total_ordenes, total_pago;
      ELSE
        RAISE EXCEPTION 'ERROR: El empleado % % % no es un
           mesero', nombre, ap_pat, ap_mat;
      END IF;
END;
$$
LANGUAGE plpgsql;
```

The stored procedure was created to display the number of orders a server has taken throughout the day, as well as the total revenue generated from those orders; if the

employee number entered does not belong to a waiter, an error is triggered.

It works by retrieving employee data from the employee table, and if the employee is a waiter, it uses aggregation functions to calculate their total and the number of sales for the day

To use this procedure, you must enter: CALL ordenes(employee_number);

**mostrar_no_disponibles**

```
------Obtener productos no disponibles
CREATE OR REPLACE PROCEDURE mostrar_no_disponibles() AS $$
DECLARE
    registro_producto record;
    contador smallint := 0;

BEGIN
    RAISE NOTICE 'A continuacion se muestran los productos no
        disponibles por el momento';
    FOR registro_producto IN
        SELECT idProducto, nombreProducto
        FROM PRODUCTO
        WHERE cantidadDisponible = 0
    LOOP
        contador := contador+1;
        RAISE NOTICE '%.->   ID: % | PRODUCTO: %',
                contador, registro_producto.idProducto,
                   registro_producto.nombreProducto;
    END LOOP;

    IF contador = 0 THEN
        RAISE NOTICE 'Todos los productos estan disponibles
           por el momento';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

This stored procedure displays the total number of products that are not available. To evaluate this event, the quantity of each product is analyzed; if it is equal to zero, its information (ID and name) is displayed. The variable registro_producto is declared

as a record, used to temporarily hold an entire row of data returned by the query during the execution of a loop. If all the products are available, a message will appear indicating this case.

To use this procedure, you must enter: CALL mostrar_no_disponibles();

### 4.3.3 Functions

Functions are used to return specific values or result sets required by the system.

**datos_factura**

```
------"Vista" para factura de una orden
--- Funcion datos_factura() que recopile los datos necesarios
    para la factura de una orden
CREATE OR REPLACE FUNCTION datos_factura(p_folio varchar(15))
RETURNS TABLE(
    idFactura integer,
    fechaFactura timestamp,
    folioOrden varchar(15),
    rfcCliente varchar(13),
    nombreCliente varchar(150),
    domicilioCliente varchar(215),
    razonSocialCliente varchar(254),
    nombreProducto varchar(50),
    cantidadProducto integer,
    precioUnitario decimal(10,2),
    precioTotalProducto decimal(10,2),
    montoTotal decimal(10,2),
    importeTotal decimal(10,2)
)
AS $$
DECLARE
    registro_factura record;

BEGIN
    RAISE NOTICE 'Generando factura para la orden %', p_folio
       ;
    FOR registro_factura IN
        SELECT f.idFactura, f.fechaFactura, f.folio, c.rfc,
```

```
            c.nomPila || ' ' || c.apPaterno || ' ' || c.
               apMaterno AS nombreCompleto,
            c.domEstado || ' ' || c.domCodigoPostal || ' ' ||
                c.domColonia || ' ' ||
            c.domCalle || ' ' || c.domNumero AS domicilio,
            c.razonSocial, p.nombreProducto, d.cantidad,
            p.precio, d.precioTotalProducto, o.totalAPagar
        FROM FACTURA f
        JOIN ORDEN o ON f.folio=o.folio
        JOIN DETALLE_ORDEN d ON d.folio=o.folio
        JOIN CLIENTE c ON c.idCliente=f.idCliente
        JOIN PRODUCTO p ON p.idProducto=d.idProducto
        WHERE f.folio=p_folio
    LOOP
        idFactura := registro_factura.idFactura;
        fechaFactura := registro_factura.fechaFactura;
        folioOrden := registro_factura.folio;
        rfcCliente := registro_factura.rfc;
        nombreCliente := registro_factura.nombreCompleto;
        domicilioCliente := registro_factura.domicilio;
        razonSocialCliente := registro_factura.razonSocial;
        nombreProducto := registro_factura.nombreProducto;
        cantidadProducto := registro_factura.cantidad;
        precioUnitario := registro_factura.precio;
        precioTotalProducto := registro_factura.
           precioTotalProducto;
        montoTotal := registro_factura.totalAPagar;
        importeTotal := (registro_factura.totalAPagar * 1.16)
           ;

        RETURN NEXT;
    END LOOP;
END;
$$ LANGUAGE plpgsql;
```

This function displays all the information needed to generate an invoice for a specific order. To achieve this, it performs a series of JOIN operations to obtain data from multiple tables. Furthermore, it uses a record variable within a loop to temporarily store data for each row, allowing to process and return the invoice details.

To use this function, you must enter: SELECT * FROM datos_factura(folioOrden);

**totales_periodo**

```
CREATE OR REPLACE FUNCTION totales_periodo(
        IN fechaInicio date,
        IN fechaFin date,
        OUT total_num_ventas smallint,
        OUT total_monto numeric(10,2)
) AS $$
BEGIN
    SELECT COUNT(*), SUM(totalAPagar)
    INTO total_num_ventas, total_monto
    FROM ORDEN
    WHERE CAST(fechaOrden AS date) BETWEEN fechaInicio AND
       fechaFin;

    IF total_monto IS NULL THEN
        total_monto := 0.00;
    END IF;
    RAISE NOTICE 'Resultados de ventas entre el % y el %',
       fechaInicio, fechaFin;
    RAISE NOTICE '>>  Numero_total_ventas: %  |
       Monto_total_ventas: $% MXN', total_num_ventas,
       total_monto;
END;
$$ LANGUAGE plpgsql;
```

This function calculates and displays business metrics for a specific time frame given in its parameters. To achieve this, it uses aggregate function such as COUNT and SUM to calculate both the total number of orders and the overall revenue.

To use this function, you must enter: SELECT * FROM totales_periodo(fechaInicio, fechaFin);

### 4.3.4 Views

Views are used to simplify access to relevant information, such as invoice-like information, unavailable products, and details of the best-selling dish.

**v_producto_mas_vendido view**

```
CREATE OR REPLACE VIEW v_producto_mas_vendido AS
SELECT p.idproducto, p.nombreproducto, p.descripcion, p.
   receta, p.precio, p.cantidaddisponible, p.tipoproducto, p.
   nombrecategoria, SUM(d.cantidad) AS total_vendido
FROM producto p JOIN detalle_orden d ON p.idproducto = d.
   idproducto
GROUP BY p.idproducto, p.nombreproducto, p.descripcion, p.
   receta, p.precio, p.cantidaddisponible, p.tipoproducto, p.
   nombrecategoria
HAVING SUM(d.cantidad) = (SELECT MAX(total_vendido) FROM (
   SELECT SUM(cantidad) AS total_vendido FROM detalle_orden
   GROUP BY idproducto) t);
```

This view is designed to display all the details of the best-selling dish or dishes. To do this, it uses a JOIN with the orders that contain the products (the ‘detalle_orden‘ table). It also uses the clauses ‘ GROUP BY‘ and ‘HAVING‘ ‘ to filter the data by dish and calculate the number of times each dish appears in all orders using a subquery.

### 4.3.5 Indexes

At least one index is created to improve query performance. The selected index must be justified according to the expected queries and the attributes most frequently used for filtering or joining data.

**idproducto index**

```
CREATE INDEX idx_detalle_orden_idproducto ON detalle_orden(
   idproducto);
```

It was decided to create an index on the ‘idproducto‘ column in the ‘detalle_orden‘ table, since it is a transactional table that will grow constantly; furthermore, it is the table that links sales (orders) with products. Furthermore, the view of the best-selling product groups data by idproducto, so PostgreSQL will need to constantly access that column, which speeds up the use of JOINs and improves the performance of reports such as sales history by product and statistical business analysis.

# 5. Presentation Layer

This section describes the selected method for connecting to the database and presenting the system functionality. The presentation layer may consist of a web application, desktop application, command-line interface, or any other mechanism chosen by the team.
The objective of this section is to explain how the user interacts with the database and how the implemented functionalities are demonstrated.

## 5.1 Data Loading from Text Files

This section describes the process used to load daily information from text files into another database. The process must include file orchestration and error validation.
The daily data generated by the restaurant's operations are exported into three main CSV text files:

- **`ordenes_dia.csv`**: Contains the daily log of customer orders, including unique folios, timestamps, total amounts due, and the assigned server's employee number.
- **`detalle_orden_dia.csv`**: Details the specific items associated with each other, tracking product identifiers, individual quantities, and calculated subtotals.
- **`facturas_dia.csv`**: Stores the billing and invoice data requested by clients, mapping corporate records to their corresponding order folios.

## 5.2 Orchestration and Workflow Design

The data loading process is managed via an ETL workflow that enforces a strict sequential execution order. This hierarchy is critical to prevent relational constraint violations, specifically regarding foreign key dependencies.
The workflow operates through the following orchestrated stages:

1. **Parent Data Ingestion:** The `ordenes_dia.csv` file is processed first. This stage establishes the parent records within the `ORDEN` table. The system reads the source text, performs data type casting, and inserts the validating records.
2. `Parallel Dependent Loading:` Once the master orders are successfully commited to the database, the pipeline triggers the parallel execution of the dependent transformations for `detalle_orden_dia.csv` and `facturas_dia.csv`. Since both target tables reference the `folio` attribute from the `ORDEN` table as a foreign key, this sequence guarantees that zero referential integrity errors occur during execution.

## 5.3 Data Validation and Error Handling

To ensure a robust environment, multiple validation checkpoints are embedded within each transformation step. The pipeline is designed to handle anomalies without disrupting the entire batch execution.
The implemented validation strategy focuses on two main pillars:

- **Data Type and Format Verification:** Before any insertion attempt, fields are strictly checked against the database schema. For instance, the system validates that numeric attributes contains true decimal formats, mandatory columns do not contain null values, and order folios strictly comply with the expected alphanumeric sequential pattern (`ORD-XXXX`).
- **Error Routing Mechanism:** Rows that fail formatting checks or violate database integrity rules are intercepted using step-level error handling. Instead of halting the process, there corrupted rows are automatically isolated and touted to a dedicated external log file (`etl_error_log.txt`. This mechanism allows administrators to analyze anomalies and re-ingest corrected data without compromising valid daily transactions.

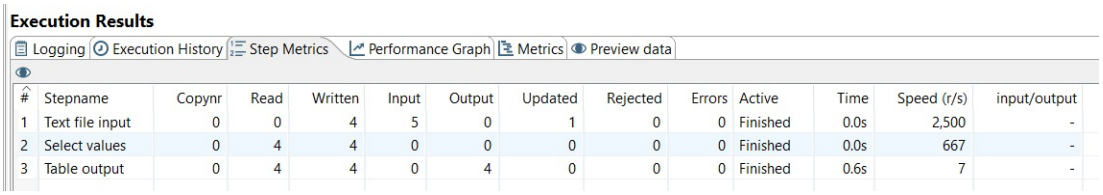
## 5.4 Execution and Verification Results

This section presents the step-by-step execution of the data integration process and the subsequent verification of the relational data inside the PostgreSQL database engine. The complete workflow validates the ETL architecture and guarantees that business rules are strictly preserved during automated daily ingestion.
The execution begins in Pentaho Data Integration (Spoon), where the orchestration workflow is triggered. Figure 3 illustrates the successful execution of the main job, showing the synchronized interaction between text file inputs, staging area updates, and final transactional insertions.

```
gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM staging_orden;
  folio   |     fechaorden      | numeroempleado | archivoorigen |         fechacarga
----------+---------------------+----------------+---------------+----------------------------
 ORD-0101 | 2026-05-30 14:20:00 |              2 |               | 2026-05-30 21:52:58.767888
 ORD-0102 | 2026-05-30 15:05:00 |              4 |               | 2026-05-30 21:52:58.767888
 ORD-0103 | 2026-05-30 16:10:00 |              2 |               | 2026-05-30 21:52:58.767888
(3 filas)
```

Figure 3: Main ETL orchestration job execution in Pentaho Spoon.

Once the global job triggers the internal transformations, each sub-process manages its own data pipeline. Figure 4 displays the execution details of the transformation responsible for reading, filtering, and preparing the daily order records before pushing them into the staging tables.



**Execution Results**

Logging | Execution History | Step Metrics | Performance Graph | Metrics | Preview data

| # | Stepname | Copynr | Read | Written | Input | Output | Updated | Rejected | Errors | Active | Time | Speed (r/s) | input/output |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Text file input | 0 | 0 | 4 | 5 | 0 | 1 | 0 | 0 | Finished | 0.0s | 2,500 | - |
| 2 | Select values | 0 | 4 | 4 | 0 | 0 | 0 | 0 | 0 | Finished | 0.0s | 667 | - |
| 3 | Table output | 0 | 4 | 4 | 0 | 4 | 0 | 0 | 0 | Finished | 0.6s | 7 | - |

Figure 4: Transformation data flow for daily orders ingestion.

Simultaneously, the billing data undergoes strict routing and structure checkups. Figure 5 presents the detailed transformation layout for the invoice records, isolating schema anomalies and preparing consistent data packets.

```
gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM staging_detalle_orden;
  folio   | idproducto | cantidad | archivoorigen |         fechacarga
----------+------------+----------+---------------+----------------------------
 ORD-0101 |          1 |        2 |               | 2026-05-30 22:49:18.491195
 ORD-0101 |          3 |        1 |               | 2026-05-30 22:49:18.491195
 ORD-0102 |          8 |        2 |               | 2026-05-30 22:49:18.491195
 ORD-0103 |          4 |        1 |               | 2026-05-30 22:49:18.491195
(4 filas)
```

Figure 5: Transformation data flow for daily invoices and error routing.

Following the completion of the staging phase, the stored procedure `cargar_staging_a_tablas_finales` is invoked at the database level to migrate data into the definitive operational model. Figure 6 confirms the successful call and execution of this migration process via the database terminal.

```
gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM orden
gpo1t_2026_2_321335630-> WHERE folio IN ('ORD-0101', 'ORD-0102', 'ORD-0103');
  folio   |     fechaorden      | totalapagar | numeroempleado
----------+---------------------+-------------+----------------
 ORD-0101 | 2026-05-30 14:20:00 |      385.00 |              2
 ORD-0102 | 2026-05-30 15:05:00 |      110.00 |              4
 ORD-0103 | 2026-05-30 16:10:00 |      170.00 |              2
(3 filas)


gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM detalle_orden
gpo1t_2026_2_321335630-> WHERE folio IN ('ORD-0101', 'ORD-0102', 'ORD-0103');
  folio   | idproducto | cantidad | preciototalproducto
----------+------------+----------+---------------------
 ORD-0101 |          1 |        2 |              190.00
 ORD-0101 |          3 |        1 |              195.00
 ORD-0102 |          8 |        2 |              110.00
 ORD-0103 |          4 |        1 |              170.00
(4 filas)


gpo1t_2026_2_321335630=>
gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM factura
gpo1t_2026_2_321335630-> WHERE folio IN ('ORD-0101', 'ORD-0102', 'ORD-0103');
 idfactura |    fechafactura     |  folio   | idcliente
-----------+---------------------+----------+-----------
         6 | 2026-05-30 14:40:00 | ORD-0101 |         1
         7 | 2026-05-30 15:30:00 | ORD-0102 |         2
(2 filas)
```

Figure 6: Execution of the relational data migration procedure in PostgreSQL.

To guarantee that data integrity was not compromised and that the triggers automatically computed operational totals, verification queries were executed on the production tables. Figure 7 shows the validated records inside the final operational tables, confirming correct structural mapping and relationship consistency.

```
gpo1t_2026_2_321335630=> SELECT *
gpo1t_2026_2_321335630-> FROM log_carga
gpo1t_2026_2_321335630-> ORDER BY fechaRegistro DESC;
 idlog |     nombrearchivo     | tabladestino  | estatus |                    mensajeerror                     |      fecharegistro
-------+-----------------------+---------------+---------+-----------------------------------------------------+-------------------------
     1 | ordenes_dia.csv       | orden         | OK      | Carga de ├rdenes finalizada. Registros insertados: 3    | 2026-05-30 23:22:29.32727
     2 | detalle_orden_dia.csv | detalle_orden | OK      | Carga de detalles finalizada. Registros insertados: 4   | 2026-05-30 23:22:29.32727
     3 | facturas_dia.csv      | factura       | OK      | Carga de facturas finalizada. Registros insertados: 2   | 2026-05-30 23:22:29.32727
     4 | proceso_carga_diaria  | general       | OK      | Proceso de carga desde staging finalizado correctamente. | 2026-05-30 23:22:29.32727
(4 filas)
```

Figure 7: Verification query testing referential integrity and final table states.

Finally, auditing mechanisms are verified to ensure comprehensive traceability. Figure 8 displays the state of the execution logs (`log_carga`), providing absolute evidence of row counts, successful steps, and automated error tracking during the operational cycle.

```
gpo1t_2026_2_321335630=> SELECT folio, totalAPagar
gpo1t_2026_2_321335630-> FROM orden
gpo1t_2026_2_321335630-> WHERE folio IN ('ORD-0101', 'ORD-0102', 'ORD-0103');
  folio   | totalapagar
----------+-------------
 ORD-0101 |      385.00
 ORD-0102 |      110.00
 ORD-0103 |      170.00
(3 filas)
```

Figure 8: Audit log table validation showcasing successful execution metrics.

# 6. Conclusions

## Personal Conclusions

- **Alvarez Salgado Eduardo Antonio**
  Throughout this project, I was able to apply several concepts studied during the course to a practical case. This allowed me to understand that database design is not only about creating tables, but also about analyzing requirements and correctly representing the logic of the system.

  One of the main challenges was interpreting certain requirements, since there were different valid ways to model them. To address this, the decisions were made by considering how the restaurant would operate in a real environment, aiming for a design that was clear, consistent, and flexible.

  Through this project, we reinforced the knowledge acquired during the course and successfully fulfilled the objectives of the practice.

- **Lara Hernández Emmanuel**

  Working on this project was a very useful experience because it allowed me to understand how a database design can be taken beyond the conceptual and relational models into a functional system with real data loading processes.

  One of the main challenges I faced was implementing the second part of the project, especially the connection between PostgreSQL and Pentaho Data Integration. Creating the staging tables, loading information from CSV files, and validating the flow before inserting the data into the final tables helped me understand the importance of having an organized ETL process.

  Overall, this project helped me improve my understanding of database administration, data loading, error handling, and the importance of keeping a clear separation between temporary data and final operational tables.

- **Merida Serralde Francisco Jared**

  Developing this restaurant management system was a very valuable experience that helped me apply the database concepts I learned in class to the development of a real-world application.

  One of the main challenges I faced during this project was correctly implementing the database programming. Specifically, creating triggers that would automatically calculate order totals while simultaneously verifying product availability required meticulous planning to avoid errors and maintain data consistency.

Overall, this project helped me improve my SQL and PL/pgSQL skills and gave me a deeper appreciation for the importance of a well-designed relational database.

- **Ponce de León Reyes Bruno**
  The design and implementation of the database for a restaurant was challenging, but it helped me better understand some important aspects of a complete project. I committed some errors while working on it; however, I was able to correct them and gained practice and understanding of how to use PL/pgSQL code. Finally, I can say developing a project of this magnitude requires great preparation and good communication among all team members to ensure that all requirements are met.

- **Zepeda Aparicio Diego Arturo**
  Developing this project helped me reinforce the importance of a well-structured database design before moving into implementation. Defining the relationships between employees, products, orders, invoices, and clients made it clear how important normalization and referential integrity are in a real system.

  One of the most important challenges was making sure that the database could satisfy the business rules requested in the problem. This required careful analysis of primary keys, foreign keys, constraints, and the order in which the information had to be inserted to avoid consistency errors.

  In conclusion, this project was valuable because it allowed me to apply the topics covered in class to a realistic restaurant management scenario. It also helped me better understand how database design decisions directly affect the reliability and maintainability of the final system.

# References

[1] PostgreSQL Global Development Group, "PL/pgSQL – SQL Procedural Language," PostgreSQL Documentation, 2025. [Online]. Available: `https://www.postgresql.org/docs/current/plpgsql.html`. [Accessed: 30-May-2026].

[2] PostgreSQL Global Development Group, "CREATE PROCEDURE," PostgreSQL Documentation, 2025. [Online]. Available: `https://www.postgresql.org/docs/current/sql-createprocedure.html`. [Accessed: 30-May-2026].

[3] PostgreSQL Global Development Group, "CREATE TRIGGER," PostgreSQL Documentation, 2025. [Online]. Available:

`https://www.postgresql.org/docs/current/sql-createtrigger.html`. [Accessed: 30-May-2026].

[4] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, U.K.: Pearson, 2015.